

La clase Object: el método clone()

La clonación es el proceso de duplicación de un objeto para que en memoria existan dos objetos idénticos (cada uno con su identidad) en el mismo instante de tiempo.



Un método de clonación devuelve un nuevo objeto cuyo estado inicial es una copia del estado actual del objeto sobre el que se invoca **clone**.

Los cambios subsiguientes en ambos objetos no afectan al otro.

La clase Object: el método clone()

La clase `java.lang.Object` contiene una implementación native y protected del método `clone`. Esta implementación determina cuánta memoria está siendo usada por el objeto a ser clonado, reserva la misma cantidad de memoria para el objeto clon, y **copia el estado del objeto original sobre el nuevo**. Y al final se devuelve un `java.lang.Object` el cual es la referencia al nuevo objeto (el clon).

Cómo se usa ?

...

```
Persona unaPersona = new Persona(....);
```

...

```
Persona otraPersona = (Persona)unaPersona.clone();
```

....

clone() - Ejemplo - consecuencias

```
public class PilaEnteros implements
    Cloneable
{ private int[] buffer;
  private int tope;

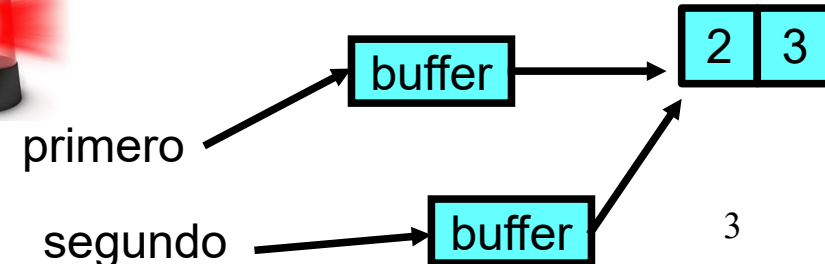
  public PilaEnteros(int contenMax)
  {   buffer = new int[contenMax];
      tope = -1;
  }

  public void meter(int val)
  {   buffer[++tope] = valor;
  }

  public int sacar()
  {   return buffer[tope--];
  }
}
```



```
PilaEnteros primera = new PilaEnteros(2);
primero.meter(2);
primero.meter(9);
PilaEnteros segunda=(PilaEnteros)primero.clone();
```



clone() - Ejemplo - solución

```
public Object clone()
{
    try
    {
        PilaEnteros nObj = (PilaEnteros)super.clone();
        nObj.buffer = (int[])buffer.clone();
        return nObj;
    }
    catch(CloneNotSupportedException e)
    {
        throw new InternalError(e.toString);
    }
}
```

Primero el método `clone` invoca a `super.clone()`

`super.clone()` puede invocar al método `Object.clone()` que crea un objeto del tipo correcto

***Try, catch
Exception ???***



Si la implementación de `clone()` de `PilaEnteros` utilizara `new` para crear un objeto `PilaEnteros`, sería incorrecto para cualquier objeto que extendiera de `PilaEnteros`



TRY, CATCH, FINALLY



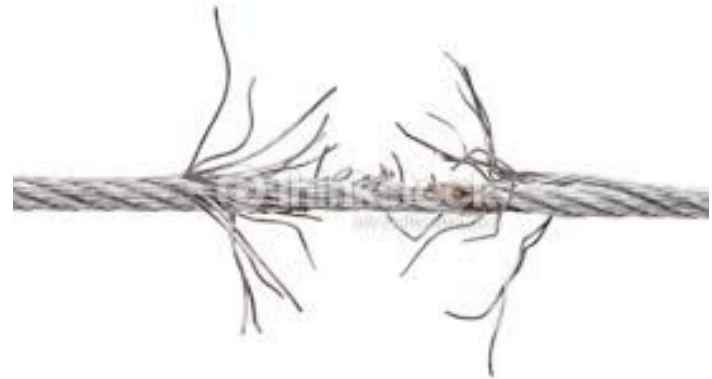
CORRESPONDE AL TEMA
EXCEPCIONES



Una excepción es un evento que ocurre durante la ejecución del programa que interrumpe el flujo normal de las sentencias. O sea, algo que altera la ejecución normal.

Muchas clases de errores pueden generar excepciones -- desde problemas de hardware, como la avería de un disco duro, a los simples errores de programación, como tratar de acceder a un elemento de un array fuera de sus límites.

$X = A / 0;$





Espacio diferente
de solución

Comprobar errores
en forma clara

Código limpio



Metodo(...)

throws Zona de propagación del error

{

try

{

Zona de ejecución normal

}

catch(Exception e)

{

Zona de recuperación

}

finally

{

Zona de finalización

}



}



Metodo(...)
{

throws

Zona de propagación del error

try

{

Zona de ejecución normal

}

catch(Exception e)

{

Zona de recuperación

}

finally

{

Zona de finalización

}

}

Dentro de la zona denominada **try**, se establecen las instrucciones que conforman el código “normal”, el que determina la funcionalidad de método. Si alguna de estas instrucciones provoca un problema (excepción), no se continúa ejecutando ninguna de las siguientes instrucciones del bloque **try** y se provoca el lanzamiento de una excepción (creación de un objeto de tipo Exception y “lanzamiento”).



Metodo(...)

{

throws

Zona de propagación del error

try

{

Zona de ejecución normal

}

catch(Exception e)

{

Zona de recuperación

}

finally

{

Zona de finalización

}

}

El lanzamiento de un objeto de tipo **Exception** altera el flujo normal del programa.

La zona **catch** se encarga de recibir por parámetro dicho objeto y en base a los datos aportados por el mismo, se elabora una estrategia de solución al problema ocurrido.



Metodo(...)

{

throws

Zona de propagación del error

try

{

Zona de ejecución normal

}

catch(Exception e)

{

Zona de recuperación

}

finally

{

Zona de finalización

}

}

Si el cuerpo del bloque try llega a comenzar su ejecución, el bloque finally siempre se ejecutará...

- Detrás del bloque try si no se producen excepciones
- Después de un bloque catch si éste captura la excepción.
- Justo después de que se produzca la excepción si ninguna cláusula catch captura la excepción y antes de que la excepción se “**propague hacia arriba**”.

Volviendo al tema...



clone() - Ejemplo - solución

```
public Object clone()
{
    try
    {
        PilaEnteros nObj = (PilaEnteros)super.clone();
        nObj.buffer = (int[])buffer.clone();
        return nObj;
    }
    catch(CloneNotSupportedException e)
    {
        throw new InternalError(e.toString);
    }
}
```

Primero el método `clone` invoca a `super.clone()`

`super.clone()` puede invocar al método `Object.clone()` que crea un objeto del tipo correcto

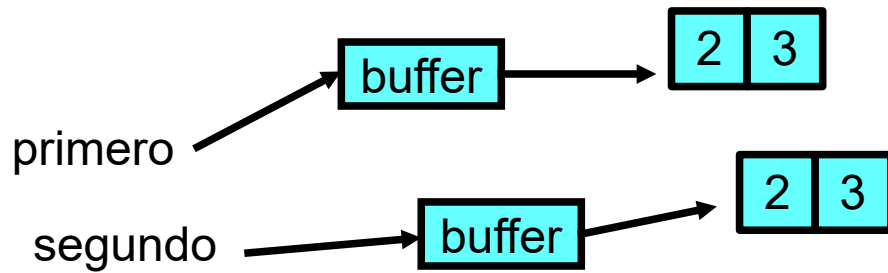


Si la implementación de `clone()` de `PilaEnteros` utilizara `new` para crear un objeto `PilaEnteros`, sería incorrecto para cualquier objeto que extendiera de `PilaEnteros`



clone() - Cómo funciona

```
public Object clone()  
{  
    try  
    {  
        PilaEnteros nObj = (PilaEnteros)super.clone();  
        nObj.buffer = (int[])buffer.clone();  
        return nObj;  
    }  
    catch(CloneNotSupportedException e)  
    {  
        throw new InternalError(e.toString());  
    }  
}
```



- ✓ `Object.clone()` inicializa todos los campos del nuevo objeto clonado asignándoles los mismos valores que los campos correspondientes del objeto que está siendo clonado.
- ✓ Sólo es necesario escribir código especial para aquellos campos en los que copiar el valor sea incorrecto.
- ✓ Los arrays pueden ser clonados.
- ✓ Si tenemos otros objetos (atributos) que no pueden ser copiados directamente, éstos deberán soportar `clone()` o deben tener un *constructor de copia*, como la clase `String`, que no soporta `clone()`, pero tiene un constructor de copia.

Actitudes frente a `clone()`:



Admitir `clone`. La clase implementa `Cloneable` y declara su método `clone` para que no lance excepciones



Admitir `clone` condicionalmente. Depende de si sus componentes pueden ser clonados. Este tipo implementará `Cloneable`, pero permitirá que su método `clone` pase cualquier excepción `CloneNotSupportedException` que pueda recibir de otros objetos que intenta clonar.



Prohibir `clone`. Una clase así no implementa `Cloneable` y proporciona un método `clone` que lanza siempre la excepción `CloneNotSupportedException`.

Estrategias de clonación

Hay tres factores a tener en cuenta:

1

La interfaz **Cloneable** que debemos implementar para proporcionar un método **clone**.

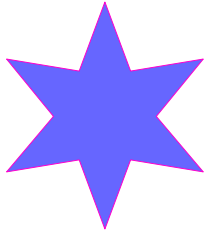
2

El método **clone** implementado por la clase **Object**, que realiza un clon simple copiando todos los campos del objeto original en un nuevo objeto.

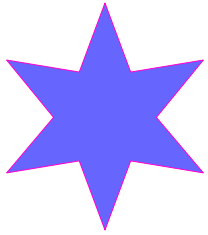
3

La excepción **CloneNotSupportedException** se puede utilizar para indicar que no se debería haber llamado a un método **clone** de una clase.

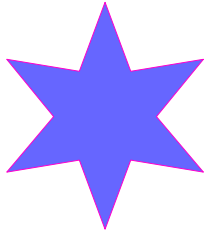
Entonces...



Las clases que no deban admitir `clone` deben tener redefinido este método, el cual debe lanzar siempre la excepción `CloneNotSupportedException`.



Puede declararse que todas las subclases de una clase deben admitir correctamente el método `clone`, redefiniendo `clone` de la clase con otro método que elimine la declaración de `CloneNotSupportedException`.



Las subclases que implementen al método `clone` no podrán lanzar la excepción `CloneNotSupportedException`, ya que los métodos de una subclase no pueden añadir excepciones a un método.